

E1. Estrategias de búsqueda y sistemas de reglas

El bot v0 funciona, pero no piensa, no planifica rutas ni dispara con criterio. En esta entrega le añadireis las primeras piezas de inteligencia básicas.

1. Búsqueda y resolución de problemas

Los algoritmos de búsqueda son herramientas de planificación, sirven cuando un agente necesita encontrar una secuencia de acciones antes de actuar. En este laboratorio diseñaréis cómo aplicar búsqueda a un problema real de Robocode. El objetivo es aprender a formular y razonar sobre un problema de búsqueda. Para ello vamos a simular un caso concreto de la batalla inaugural.

Vuestro bot está en la posición (0,0), la esquina superior izquierda, con energía baja. El rival se encuentra en la posición (3,7). Si sigue quieto, lo embisten. Si huye en línea recta hacia la derecha, cruza la zona de peligro. La pregunta es: ¿cuál es el camino más seguro para escapar al refugio (8,3)?

1.1. La arena y la función de coste

La arena está modelada como una rejilla de 10×10 celdas. Cada celda tiene un coste de entrada que representa el peligro de pasar por ella.

Vamos a trabajar sobre el siguiente mapa donde B = bot (0,0) R = rival (3,7) * = refugio (8,3). Cada número es el coste de entrar en esa celda.

	0	1	2	3	4	5	6	7	8	9
0	B	4	4	4	4	4	4	4	4	7
1	4	1	1	2	1	1	1	1	1	4
2	5	2	2	2	2	2	2	1	1	4
3	5	2	2	5	2	2	2	2	*	4
4	5	5	5	5	5	5	2	2	2	4
5	8	5	5	9	5	5	5	2	2	4
6	8	5	9	9	9	5	5	2	2	4
7	8	9	9	R	9	9	5	5	2	5
8	8	5	9	9	9	5	5	2	2	4
9	11	8	8	12	8	8	8	5	5	7

La función de costes de este mapa se ha definido con las siguientes condiciones:

CONDICIÓN	COSTE	RAZÓN
Celda interior sin peligro especial	+1 (base)	Toda celda cuesta al menos 1
Rival a entre 5 y 6 de distancia	+1 (anillo exterior)	Zona de alerta: el rival puede llegar pero no es inminente
Rival a entre 3 y 4 de distancia	+4 (anillo medio)	Zona de precaución: rango de disparo frecuente
Rival a 2 o menos de distancia	+8 (zona de peligro)	Zona crítica: disparo seguro y riesgo de embestida
Celda en el borde izquierdo o derecho (col 0 ó 9)	+3	Contra la pared no hay escapatoria lateral
Celda en el borde superior o inferior (fila 0 ó 9)	+3	Ídem para los bordes horizontales

Como se puede observar en el mapa, los costes se suman. Por lo que una celda de borde inferior pegada al rival puede llegar a coste 12. El coste de entrar en una celda es siempre mayor o igual que 1. La distancia al rival en el mapa se ha calculado con la fórmula de la distancia euclídea:

$$d = \sqrt{(col_{rival} - col_{celda})^2 + (fila_{rival} - fila_{celda})^2}:$$

- Verificad el coste de al menos 3 celdas a mano, aplicando la tabla de coste. Mostrad el cálculo paso a paso.

1.2. Formulación del problema de búsqueda

Con el mapa y la función de coste ya dados, formalizad el problema definiendo estos cinco elementos clave:

ELEMENTO	VUESTRA DEFINICIÓN
Estado	¿Cómo representáis una posición en el mapa?
Estado inicial	¿Dónde empieza la búsqueda?
Acciones	¿Cuáles son los movimientos posibles del bot?
Prueba de objetivo	¿Cuándo ha llegado el bot al refugio?
Coste de cada acción	¿Cuánto cuesta moverse de una celda a otra?

1.3. Búsqueda no informada

Una vez definido el problema, es momento de buscar la solución. Primero analizaréis la solución utilizando uno de los algoritmos de búsqueda no informada (BFS, búsqueda en profundidad, coste uniforme...).

Elegid uno de los algoritmos de búsqueda no informada del tema y justificar:

- ¿Por qué habéis elegido ese algoritmo y no otro?
- ¿Garantiza encontrar el camino de menor coste?
- ¿Cuál es el punto débil del algoritmo en este problema concreto?

Ahora ejecutad el algoritmo elegido sobre el mapa proporcionado en 1.1, desde (0,0) hasta (8,3). Documentad la ejecución en una tabla o en un árbol, similar a los ejemplos vistos en clase. Por último dad el camino resultante, el coste total, y los nodos expandidos.

1.4. Búsqueda informada

Ahora repetid el proceso seleccionando uno de los algoritmos de búsqueda informada (A* o búsqueda voraz).

Definid la heurística que vais a usar. ¿Es admisible? Razonad la respuesta.

Ejecutad el algoritmo elegido con vuestra heurística, de nuevo desde (0,0) hasta (8,3). Documentad la ejecución (tabla o árbol) y dad el camino resultante, el coste total y los nodos expandidos.

1.5. Comparación de algoritmos

1. ¿Los dos algoritmos encuentran el mismo camino? ¿Tienen el mismo coste? Si difieren, ¿cuál es mejor y por qué?
2. ¿Cuál ha explorado menos nodos? ¿Qué implica eso para un bot con límite de tiempo por turno?
3. El camino más directo sería ir por la fila 0 hasta la columna 8 y bajar a (8,3). Calculad su coste a mano. ¿Es peor respecto al camino de vuestro algoritmo informado?
4. ¿Tiene sentido implementar este sistema en el bot real? ¿Qué problema habría si el rival se mueve entre el momento en que se calcula la ruta y el momento en que se sigue?

2. Representación del conocimiento

En el apartado anterior usasteis búsqueda para planificar la retirada más segura calculando la ruta completa antes de actuar. Pero en un combate real el rival se mueve cada turno, recalculad el algoritmo de búsqueda continuamente puede consumir demasiado tiempo y superar el límite por turno. Los sistemas basados en reglas ofrecen una alternativa complementaria, en vez de planificar, el bot reacciona, evaluando la situación en cada turno y aplica la regla que corresponde, en tiempo constante.

Un sistema de reglas tiene tres componentes: la base de conocimiento (las reglas IF-THEN que codifican qué hacer en cada situación), los hechos (lo que percibe el bot en ese turno: distancia al rival, energía, calor del cañón...) y el motor de inferencias (el bucle del bot que evalúa las reglas y ejecuta la que aplica). En Robocode, los manejadores `on_scanned_bot` y `on_tick` hacen de motor de inferencias.

2.1. ¿Qué sabe el bot en cada turno?

Para diseñar reglas necesitáis saber con qué información contáis en cada momento. El bot percibe el mundo a través de dos vías:

- **Siempre disponible** (en cada turno, sin necesidad de escanear): `self.energy`, `self.x`, `self.y`, `self.gun_heat`, `self.speed`, `self.direction`, `self.gun_direction`, `self.radar_direction`.
- **Solo cuando el radar escanea** (en `on_scanned_bot`): posición del rival (`e.x`, `e.y`), su energía (`e.energy`), velocidad (`e.speed`), dirección (`e.direction`), distancia (`self.distance_to(e.x, e.y)`) y bearing del cañón al rival (`self.gun_bearing_to(e.x, e.y)`).

Importante: entre escaneos el bot no sabe dónde está el rival. Por eso es imprescindible guardar la última información conocida en variables de instancia (`self._rival_x`, `self._rival_dist`...) para poder usarla en las reglas aunque no haya un escaneo en ese turno.

2.2. Eventos

Además de lo que sabéis por estado y por escaneo (apartado 2.1), Robocode os avisa activamente cuando os ocurre algo mediante **eventos**: os disparan, chocáis, muere un rival. Cada evento dispara un método `on_*` que ya conocéis de vuestro v0.

Una regla que reacciona a un evento es tan legítima como una que reacciona a la distancia o la energía: podéis escribir `SI on_hit_by_bullet ENTONCES girar 45º` igual que escribís reglas sobre `self.energy`. La diferencia es que estas se activan una vez, en el turno del evento, no en cada turno.

Os recuerdo algunos eventos de Robocode a continuación:

Evento	Qué os dice	Reacción típica
<code>on_hit_by_bullet(e)</code>	Os ha acertado una bala. <code>e.energy</code> es vuestra energía tras el impacto, <code>e.bullet.direction</code> de dónde venía.	Girar y moveros para no seguir en la misma línea de tiro.
<code>on_hit_wall(e)</code>	Habéis chocado con el borde de la arena.	Invertir el sentido de la marcha antes de seguir empujando.
<code>on_hit_bot(e)</code>	Habéis chocado con otro bot. <code>e.energy</code> es la vuestra tras el choque.	Separaros o embestir de nuevo, según vuestra energía relativa.
<code>on_bullet_hit(e)</code>	Una bala vuestra ha acertado a un rival.	Normalmente ninguna: es informativo, útil para registrar puntería.
<code>on_bot_death(e)</code>	Ha muerto un rival (<code>e.victim_id</code>).	Dejar de apuntarle si era vuestro objetivo; el radar buscará otro.

Movimientos básicos de la comunidad Robocode:

- **Esquiva tras impacto:** al recibir una bala, girar en un ángulo (p. ej. 90°) antes de seguir moviéndose, para no ofrecer la misma trayectoria al siguiente disparo.
- **Movimiento en zigzag:** alternar periódicamente el sentido de avance en vez de ir siempre recto; un blanco que se mueve en línea recta es el más fácil de acertar.
- **Rebote en paredes:** invertir el sentido al chocar (regla que ya usa vuestro `v0`) en vez de quedaros empujando contra el borde, donde sois un blanco fácil e inmóvil.
- **Círculo alrededor del rival (`circle strafing`):** combinar avance y giro constante para manteneros en movimiento relativo al rival mientras coincidís con la potencia de disparo.
- **Mantener distancia de seguridad:** si el rival se acerca demasiado, alejarse; si está muy lejos, perderéis precisión pero también recibiréis menos disparos.

No es obligatorio usar ninguna de estas estrategias, son punto de partida o ideas. Lo importante es que vuestras reglas cubran tanto lo que **veis** (escaneo) como lo que **os pasa** (eventos), que es justo lo que os pedirá la tabla de reglas del apartado siguiente.

2.3. Diseño de reglas

Diseñad en una tabla vuestro sistema de reglas (**mínimo 6, máximo 12**) en formato `SI <condición> ENTONCES <acción> PRIORIDAD <n>`. Deben cubrir al menos:

- Qué hacer cuando no veis al rival (regla por defecto).
- Cuándo y con qué potencia disparar (según distancia y calor del cañón).
- Cuándo acercarse o alejarse.

La retirada: en el apartado 1 razonasteis cuál era la retirada más segura sobre la rejilla. Ese mapa es una simplificación en papel: la arena real es continua y el rival se mueve, así que el bot no va a seguir esa ruta celda a celda. Lo que sí se traslada es la idea, cuándo dejar de combatir y hacia dónde huir. Definid una regla del tipo `SI energía < X ENTONCES alejarse hacia una zona segura`, por ejemplo la esquina más alejada del rival, y justificad el umbral `X` que elijáis.

¿Y qué números tienen sentido para los umbrales?

Esto lo analizaréis con vuestros propios datos en E2. Por ahora, comprobad que estáis dentro de estos rangos posibles de las variables importantes, y justificad el umbral concreto que elijáis.

Variable	Rango posible	De dónde sale
<code>self.energy</code>	0 a 100	Empieza en 100; en 0 o menos el bot está derrotado. Un <code>SI energía < 150</code> no se cumple nunca.
<code>self._rival_dist</code>	0 a ≈ 1000	Diagonal de una arena de 800×600 (tamaño por defecto de la GUI).
Potencia de disparo	0,1 a 3	<code>set_fire()</code> ignora valores fuera de ese rango.
<code>self.gun_heat</code>	0 a $\approx 1,6$	Empieza a 3,0 y se enfría cada turno; el calor máximo tras disparar a potencia 3 son $\approx 1,6$ puntos.

2.4. Resolución de conflictos y completitud

Cuando dos reglas se activan a la vez hay que elegir. Definid la estrategia de resolución de conflictos que usáis (prioridad, especificidad o más reciente) y justificadla.

Analizad el caso concreto: ¿qué ocurre si en el mismo turno se activan "disparar" y "retirarse"?

En Robocode, disparar y moverse son acciones que se ejecutan simultáneamente en el mismo turno: `set_fire()` y `set_forward()` no se excluyen mutuamente. Por tanto, el conflicto no es físico (el bot PUEDE hacer ambas cosas a la vez) sino estratégico: ¿merece la pena gastar energía en un disparo cuando el bot ya está bajo de vida? Tened en cuenta que disparar consume entre 1 y 3 puntos de energía según la potencia elegida.

Razonad vuestra decisión: si el bot tiene poca energía y el rival está cerca con el cañón frío, ¿priorizáis la retirada (conservar energía) o disparáis antes de huir (un impacto a corta distancia hace más daño del que cuesta)? No hay una respuesta única, pero debéis argumentar la vuestra con números.

Demostrad también que vuestro sistema es completo: para cualquier situación posible, siempre hay al menos una regla aplicable. ¿Tenéis una regla por defecto que se active cuando ninguna otra lo hace?

2.5. Antes de programar v1

Recordad de la Introducción que cada acción tiene una versión **bloqueante** (`forward`, `turn_right`, `fire...`) y otra **no bloqueante** con prefijo `set_` (`set_forward`, `set_turn_right`, `set_fire...`). La bloqueante ejecuta esa acción y llama a `go()` por vosotros; si encadenáis dos llamadas bloqueantes, ocurren en turnos distintos, una detrás de otra. El esqueleto proporcionado aquí usa siempre las versiones `set_` dentro de `run()` y cierra el turno con un único `go()` al final, nunca con las bloqueantes. Pero si consideráis necesario cambiarlo, es vuestra decisión.

Ejemplos no bloqueantes:

Método	Qué hace
<code>set_forward(d)</code>	Avanza d unidades
<code>set_turn_right(g)</code>	Gira el cuerpo g grados
<code>set_turn_gun_right(g)/ set_turn_gun_left(g)</code>	Giran el cañón

Apuntar a un punto

Para esquivar una bala (visto en la Introducción) bastaba con `calc_bearing` sobre la dirección de la bala. Para apuntar a un punto se utilizan las funciones de la familia `bearing_to`.

Método	Qué devuelve
<code>direction_to(x, y)</code>	La dirección absoluta desde vuestra posición hasta (x, y).
<code>bearing_to(x, y)</code>	Cuánto girar el cuerpo desde su dirección actual para apuntar directo a (x, y).
<code>gun_bearing_to(x, y)</code>	Lo mismo para el cañón.
<code>radar_bearing_to(x, y)</code>	Lo mismo para el radar.

El signo del giro

En Tank Royale, un giro positivo gira a la izquierda y uno negativo gira a la derecha.

Vuestro propio sentido

Si necesitáis una variable propia para saber en qué sentido vais, del tipo:

```
self._sentido = 1          # se inicializa en __init__
self._sentido = -self._sentido # se invierte cuando toque
```

De modo, igual que con el rival, el bot necesita memoria de sí mismo entre turnos para cualquier comportamiento que no dependa solo del estado actual.

Condición de ver al rival

Para calcular si hemos visto al rival hace poco, vamos a usar:

```
visto = (self._rival_dist is not None and
         self.turn_number - self._ultimo_escaneo < N)
```

El valor de `N` lo podéis modificar en función de cómo reciente queréis que sea. `N=1` significa que solo contáis el rival como visto en el mismísimo turno del escaneo; un turno después ya no vale, así que en la práctica casi nunca dispararíais. Cuanto mayor sea `N`, más margen dais, pero también más riesgo de apuntar a una posición que ya no es la real.

Recordatorio: girar el cañón no gasta energía ni depende del calor, solo disparar lo necesita. Por eso el esqueleto separa las dos cosas dentro del grupo cañón: apuntar solo necesita ser visto, y disparar espera además a que el cañón se haya enfriado (`if self.gun_heat == 0`)

2.6. Implementación de v1

Traducid las reglas a Python preparando la versión v1 de vuestro bot. La clave está en separar los tres componentes del sistema, que hasta ahora habéis descrito en papel, en dos sitios distintos del código: `on_scanned_bot` solo actualiza los **hechos** y no decide nada, y `run()` es el **motor de inferencias**, que cada turno evalúa las reglas con los hechos disponibles.

Dentro de `run()` usad `if/elif`, de forma que solo se aplique una regla y el orden de los `if` codifique la prioridad. Podéis agrupar por tipo de acción (movimiento, cañón, radar..). Comentad cada bloque con su número (# R1, # R2...).

Podéis utilizar este esqueleto y añadir todas las reglas de vuestra tabla como corresponda, enumerándolas debidamente.

```
def __init__(self):
    super().__init__("VuestroBot.json")
    self._rival_x = self._rival_y = self._rival_dist = None
    self._ultimo_escaneo = 0

def on_scanned_bot(self, e: ScannedBotEvent):
    # HECHOS: aquí solo se percibe, no se decide
    self._rival_x, self._rival_y = e.x, e.y
    self._rival_dist = self.distance_to(e.x, e.y)
    self._ultimo_escaneo = self.turn_number

def run(self):
    while self.is_running:
        visto = (self._rival_dist is not None and
                 self.turn_number - self._ultimo_escaneo < N)

        # Una sola regla gana (if/elif/else = orden de prioridad)
        if self.energy < X:                    # R1 retirada
            ...
        elif visto and self._rival_dist > Y:    # R2 acercarse
            ...
        elif visto and self._rival_dist < Z:    # R3 alejarse
            ...
        else:                                  # R4 por defecto

        # Además puede dispararse el mismo turno en que os movéis.
        if visto:
            ...                                # apuntar al rival
            if self.gun_heat == 0:
                if self._rival_dist < A:        # R5 potencia alta
                    self.set_fire(3)
                elif self._rival_dist < B:      # R6 potencia media
                    self.set_fire(2)
                else:                           # R7 potencia baja
                    self.set_fire(1)

        self.set_turn_radar_right(45)          # barrer el radar SIEMPRE

        self.go()                              # cierra el turno
```

Entrega

Guión de la memoria (máximo 10 páginas)

1. **Verificación del mapa:** verificación de al menos 3 celdas con cálculo paso a paso e identificación de la zona de mayor riesgo.
2. **Formulación del problema:** los 5 elementos con justificación de los movimientos posibles del bot.
3. **Búsqueda no informada:** algoritmo elegido, respuesta a las preguntas de justificación y tabla de ejecución con camino, coste y nodos expandidos.
4. **Búsqueda informada:** heurística definida, comprobación de admisibilidad, algoritmo elegido y tabla de ejecución con camino, coste y nodos expandidos.

5. **Comparación de algoritmos:** respuestas a las preguntas del apartado 1.5.
6. **¿Qué sabe el bot?** identificación de los hechos usados en las reglas, justificación de por qué cada uno es relevante, y justificación del valor de N elegido para la condición de "visto".
7. **Tabla de reglas:** (mínimo 6 y máximo 12), en formato SI/ENTONCES/PRIORIDAD con el umbral de cada una justificado y el propósito de cada una.
8. **Resolución de conflictos y completitud:** estrategia elegida, análisis del caso "disparar vs retirarse" y demostración de completitud.
9. **Evaluación v0 vs v1:** comparación y análisis.
10. **Declaración del uso de la IA**

Además de la **memoria** debéis entregar el .zip (.py, .json y scripts) del bot v1 en Aula Global. No hace falta incluir datos_batallas/.